Intelligent Spring Boot CI/CD Assistant

Project Id: 25-26J-451

Project Proposal Report

Senevirathna K.M.U.T

B.Sc. (Hons) Degree in Information Technology Specialized in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology Sri Lanka

August 2025

Intelligent Spring Boot CI/CD Assistant

Project Id: 25-26J-451

Project Proposal Report

B.Sc. (Hons) Degree in Information Technology Specialized in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology Sri Lanka

August 2025

DECLARATION

We declare that this is our own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Declaration

We declare that this is our own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature
Senevirathne KMUT	IT22056320	Qudula Thathsarada

Signature of the Supervisor:
Date: 28/08/2025

Signature of the Co-Supervisor: S.S Kumarasinghe 27/08/25
Date:

RESEARCH ABSTRACT

In modern enterprise development, manual creation of Dockerfiles, Kubernetes manifests, and CI/CD pipelines for Spring Boot projects is time-consuming, error-prone, and often insecure. Developers spend significant effort configuring deployment artifacts, while security issues such as hard-coded secrets, missing resource limits, and root-level containers remain common.This project proposes an IntelliJ IDEA plugin called the Intelligent Spring Boot CI/CD Assistant. It leverages Claude 4 Sonnet via Amazon Bedrock, combined with Model Context Protocol (MCP) for live project context extraction and prompt engineering for secure artifact generation. Unlike existing tools, it not only generates Docker/Kubernetes/CI artifacts but also integrates multi-layer security validation (Hadolint, KubeLinter, Spring Boot rules) and provides line-by-line explainability to educate developers. PostgreSQL is used for audit logs and analytics, ensuring traceability and compliance.The system aims to reduce configuration time by ≥50%, achieve ≥80% first-run deployment success, and improve developer onboarding by providing transparent explanations of generated infrastructure.

Keywords: Spring Boot, IntelliJ plugin, CI/CD automation, Claude 4 Sonnet, Bedrock, MCP, Prompt Engineering, DevSecOps, Hadolint, KubeLinter, Explainable AI.

Table of Contents

DECLARATION i

Table of Contents iii

List of Figures iv

List of Tables v

List of Abbreviations vi

1. INTRODUCTION i

1. 1 Background & Literature Survey i

Literature Survey ii

1. 2 Research Gap v

1. 3 Research Problem ix

2. OBJECTIVES xii

2. 1 Main Objectives xii

2. 2 Specific Objectives xii

3. METHODOLOGY xiii

3. 1 System Architecture xiii

3. 2 Software Solution xv

4. PROJECT REQUIREMENTS xix

4. 1 Functional Requirements xix

4. 2 Non-functional requirements xx

4. 3 System requirements xx

4. 4 User requirements xxi

5. Gantt Chart xxii

5. 1 Work Breakdown Structure (WBS) xxii

6. BUDGET AND BUDGET JUSTIFICATION xxiii

REFERENCES xxiv

List of Figures

Figure 1.1 Comparison with Existing Systems ix
Figure 3.1 Intelligent Spring boot AI Assistant System Architecture xiii
Figure 5.1:Gantt Chart xxii

List of Tables

Table 1.1 Research Gap Analysis viii
Table 6.1 Budget Justification table xxiii

List of Abbreviations

Abbreviation	Word/Description
ML	Machine Learning
CI/CD	Continuous Integration / Continuous Deployment
IDE	Integrated Development Environment
IaC	Infrastructure as Code
LLM	Large Language Model
MCP	Model Context Protocol

1. INTRODUCTION

1. 1 Background & Literature Survey

Spring Boot has become one of the most widely adopted frameworks for building Java applications due to its opinionated configurations, microservice support, and seamless integration into enterprise systems [7][16]. According to the JetBrains Developer Ecosystem 2024 survey, more than 65% of enterprise Java developers rely on Spring Boot for backend services [17]. Its popularity stems from rapid setup, simplified dependency management, and strong support for microservice architectures.

However, while Spring Boot streamlines application development, the DevOps layer remains complex and resource-intensive. Developers must still manually define and maintain Dockerfiles for containerization, Kubernetes manifests for orchestration, and YAML-based CI/CD pipelines (e.g., GitHub Actions, Jenkins, GitLab CI) for automation [12][13]. These tasks demand DevOps expertise, which many Java-focused teams lack, leading to configuration errors, delays, and inconsistent deployment quality.

Security is another major challenge. Studies show that nearly 40% of infrastructure code, whether manually authored or AI-generated, contains vulnerabilities such as hardcoded secrets, missing resource limits, or root-level container execution [2][4]. Misconfigurations further exacerbate risks by creating insecure deployments that may fail compliance checks. In addition, configuration drift frequently arises as projects evolve, causing infrastructure definitions to fall out of sync with actual application requirements [5]. These gaps can lead to failed deployments, performance bottlenecks, or even production outages.

The productivity impact is also significant. Enterprises report developers spending 12–15 hours per week maintaining deployment code instead of building business logic [1][18]. This inefficiency directly affects delivery speed and innovation capacity. With the DevOps tools market valued at USD 8.2 billion in 2023 and expected to grow at a 20% CAGR [18], organizations

are increasingly seeking solutions that combine automation, security, and context-awareness.

Yet existing solutions remain fragmented. Tools like Spring Initializr scaffold projects but stop short of generating deployment pipelines. CI/CD orchestrators such as Jenkins and Argo CD are powerful but require manual configuration. AI-based tools like GitHub Copilot provide code suggestions but lack Spring Boot awareness, security validation, and explainability [3][6]. This leaves developers without an integrated, framework-specific solution that reduces manual effort while ensuring security and compliance.

The proposed Intelligent Spring Boot CI/CD Assistant addresses this gap by offering an IntelliJ IDEA plugin that combines:

- Claude 4 Sonnet (via AWS Bedrock) for artifact generation and explainability,
- Model Context Protocol (MCP) for extracting live project metadata, and
- multi-layer security validation using Hadolint, KubeLinter, and Spring Boot compliance rules [14][15][16].

By embedding these capabilities directly in the developer's IDE, the assistant aims to reduce manual configuration effort by 50%, minimize security risks, and improve both developer onboarding and knowledge transfer.

Literature Survey

Several strands of research and industrial practice inform this work at the intersection of artificial intelligence, DevSecOps, and IDE integration. Rapid advances in AI coding assistants have opened new opportunities while exposing gaps in framework-specific automation and security compliance.

AI for code and infrastructure generation

Early evidence showed that large language models can generate useful code. Chen et al. reported strong results for Codex on benchmarks such as HumanEval, while noting limitations in framework-specific scenarios and no capability for infrastructure

artifacts such as Dockerfiles or Kubernetes manifests [1]. Wang et al. introduced CodeT5, which improves context awareness through identifier information and delivers better performance on code understanding and generation, yet remains general purpose rather than DevOps focused [2]. Meta's Code LLaMA family extends open models for programming tasks and achieves competitive scores on HumanEval and MBPP, offering an adaptable base for domain applications [6][7][8]. More recent work on deployability-centric IaC shows that combining model guidance with domain constraints can push success rates above 80 percent for Terraform template generation, indicating the value of validation-aware approaches [5]. These lines of work support the premise that LLMs can draft infrastructure code but require context infusion and post-generation checks before use in enterprise pipelines.

Security in AI-generated infrastructure

Security concerns are well documented. Perry et al. found that about 40 percent of AI-generated samples in critical settings contain vulnerabilities, including improper validation and insecure cryptographic choices [3]. A systematic review of Infrastructure as Code repositories reports that 78 percent include serious misconfigurations, such as hardcoded secrets, missing resource bounds, and permissive access controls [4]. Best practice guidance from Docker and the CNCF emphasizes version pinning, non-root execution, resource constraints, probes, and modern API usage [12][13]. Tooling has matured to operationalize these practices. Hadolint enforces Dockerfile rules programmatically [14], KubeLinter validates Kubernetes YAML for common risks [15], and Trivy adds image and configuration vulnerability scanning [20]. OWASP guidance on prompt safety further motivates controlled context injection when models are used inside build systems [11]. Together these results support a design that always pairs generation with automated validation.

IDE-integrated AI assistants

Commercial assistants improve developer productivity but stop short of infrastructure automation. GitHub Copilot, powered by Codex, excels at rapid code completions in file-level contexts yet does not provide security validation or framework-specific

deployment knowledge [1][3]. JetBrains AI Assistant integrates multiple models and benefits from deep IDE awareness, which helps multi-file reasoning, but its scope centers on source code assistance rather than CI or platform artifacts [17]. Spring tooling in IntelliJ, including Spring Initializr and configuration helpers, streamlines application setup but does not generate Docker, Kubernetes, or pipeline descriptors. This landscape indicates an opening for an IDE-native assistant that is aware of Spring Boot conventions and couples generation with validation and explanations.

DevOps and CI/CD automation practices

Jenkins, GitHub Actions, Azure DevOps, and Argo CD remain foundational in enterprise delivery, yet they rely on manually authored pipelines and templates [19]. Industry reports note that teams spend substantial time maintaining deployment definitions for Spring Boot systems, often 12 to 15 hours weekly, which diverts effort from product features [18]. Practice guides show that well structured pipelines and container builds can reduce lead time by large margins, but only when best practices such as multi-stage builds, health checks, and resource limits are applied consistently [12][13]. These observations support the need for automation that is both context aware and policy enforcing.

Domain-specific automation and opportunities

Recent work demonstrates the feasibility of targeted automation for specific stacks. Deployability-driven IaC generation improves success by integrating constraint checks into the loop [5]. Experimental efforts around Spring projects show benefits from model-assisted testing and pipeline integration, suggesting that a Spring-aware assistant can deliver meaningful gains when it understands framework structure and deployment patterns. The remaining gap is to bring these capabilities into the IDE, attach them to live project context, and guarantee security checks before outputs are accepted.

IntelliJ plugin foundation

The IntelliJ Platform provides APIs for project analysis, background tasks, external service calls, and UI integration, which are suitable for an assistant that reads project metadata, orchestrates external validators, and displays explanations. Existing Spring plugins enhance development experience but do not address deployment artifact generation. This platform therefore offers a practical foundation for an IDE-native CI/CD assistant that integrates project context, model generation, and validation in one workflow.

Synthesis and identified gaps

The literature indicates three consistent gaps. First, LLMs can generate code but lack Spring Boot specific deployment knowledge and cannot guarantee secure defaults on their own [1][2][6][7][8]. Second, security tooling is effective but remains separate from everyday authoring, which raises the barrier to consistent use [12][13][14][15][20]. Third, CI/CD platforms automate execution but still require manual pipelines and do not teach or explain choices to developers [18][19]. The proposed Intelligent Spring Boot CI/CD Assistant addresses these gaps by combining IDE integration, live context extraction through a model context protocol, multi-layer validation with Hadolint, KubeLinter, and Spring Boot rules, and explanation of each generated artifact. This configuration positions the assistant to reduce setup time, raise security compliance, and improve developer learning within a familiar IntelliJ workflow.

1. 2 Research Gap

The reviewed literature demonstrates significant progress in AI-assisted code generation, infrastructure automation, and IDE integration. However, several critical gaps remain unaddressed, particularly in Spring Boot enterprise development, where secure, explainable, and efficient automation is essential. These gaps span technical, security, usability, and integration domains:

1. Limited Spring Boot–Specific Knowledge in AI Models

AI models such as OpenAI Codex and CodeT5 have demonstrated general-purpose code generation capabilities [1][2]. However, they lack domain-specific expertise in Spring Boot, often producing syntactically correct but semantically inconsistent

artifacts that violate framework conventions (e.g., layered architecture, dependency injection). Even advanced open models like Code LLaMA show promise [6][7] but remain underutilized for Spring Boot–specific infrastructure generation. No current LLM is fine-tuned for Spring Boot pipelines, leaving a gap for enterprise-ready, framework-aware assistants.

2. Missing Integrated Security Validation

Research consistently highlights security weaknesses in AI-generated code: Perry et al. (2022) showed nearly 40% of outputs contain vulnerabilities [3], while Kumar et al. (2022) reported 78% of IaC repositories misconfigured [4]. Despite mature tools such as Hadolint [14] and KubeLinter [15], none are natively integrated into AI-driven assistants. Moreover, Spring Boot–specific security practices—such as enforcing non-root container execution, secure profile handling, and compliance with Spring Security defaults—remain unaddressed. This creates a critical gap where generation happens without automatic validation, risking insecure deployments.

3. Lack of Explainability in Infrastructure Generation

Tools like GitHub Copilot and JetBrains AI Assistant improve productivity but offer minimal explainability [17]. Developers, especially juniors, often receive YAML pipelines or Dockerfiles without line-by-line justifications, slowing onboarding and reducing trust. This is especially problematic in enterprise DevSecOps, where developers must understand why certain resource limits, probes, or secrets are configured. No existing assistant combines generation with explanations and validation inside the IDE.

4. Fragmented Lifecycle Coverage

Domain-specific automation efforts remain narrow. For example, Zhang et al. (2025) demonstrated deployability-centric IaC generation for Terraform [5], while projects like CUBETESTERAI focused on JUnit test automation for Spring Boot. Yet, these efforts target isolated tasks rather than the end-to-end DevOps workflow. Existing tools do not provide Spring Boot–aware CI/CD pipelines that are secure, validated, and explainable within a single workflow.

5. Challenges in Using Live Project Context

Effective automation requires real-time awareness of project metadata—dependencies, configs, ports, annotations, and architectural style. Current AI assistants lack

mechanisms for dynamic context injection. The Model Context Protocol (MCP), introduced in 2024, provides a standard for safe context sharing, but adoption is limited, and no existing tool integrates MCP with Spring Boot development. Without it, assistants generate generic and incompatible infrastructure artifacts that need manual rework.

6. Scalability and Developer Experience Issues

Enterprise Spring Boot systems typically involve multi-module microservices, diverse environments, and complex pipelines. Existing assistants cannot scale pipelines dynamically (e.g., adjust builds, apply multi-stage Docker optimizations, minimize redundancies). Nor do they provide intuitive IDE-native controls for developers to review, customize, and validate generated artifacts without breaking workflow.

7. Compliance and Enterprise Readiness

Most AI code generation research ignores regulatory compliance (GDPR, HIPAA) and accessibility (WCAG). For enterprises adopting DevSecOps at scale, this is a blocker. No current tool enforces compliance rules or supports audit logging within IDE-driven AI assistants. This gap reduces trust and prevents enterprise adoption.

Summary Table of Key Gaps

Gap Area	Description	Impact
Framework-Specific Code	Lack of Spring Boot–aware training and adaptation in AI models	Suboptimal, non-compliant code generation requiring manual fixes
Security Validation	Missing integrated multi-layer security checks for Docker, Kubernetes, and Spring Boot specifics	High vulnerability risk and compliance failures
Explainability & DevOps Coverage	Limited explainability of generated IaC and missing DevOps artifact generation	Poor developer understanding and fragmented workflows

Domain-Specific Automation Scope	Narrow focus on either infrastructure or testing automation	Lack of end-to-end deployment pipeline automation
Live Project Context Extraction	Inability to incorporate live project metadata and context into generation	Incompatible and inefficient code artifacts
Scalability & Developer UX	Lack of pipeline optimization and seamless IDE integration	Reduced developer productivity and increasing operational costs
Compliance & Accessibility	No integration of regulatory and accessibility standards	Limits enterprise adoption and legal compliance

Table 1.1 Research Gap Analysis

These multidimensional gaps highlight the need for an IntelliJ-native assistant that combines:

- Deep Spring Boot context awareness (via MCP).
- Secure, explainable CI/CD pipeline generation (Claude 4 Sonnet + validation).
- Integrated validation with Hadolint, KubeLinter, Spring Boot rules.
- Developer education through line-by-line explanations.
- Audit logs and compliance tracking via PostgreSQL.

The proposed Intelligent Spring Boot CI/CD Assistant directly addresses these unmet needs, providing the first IDE-embedded solution that unifies context extraction, AI-powered artifact generation, validation, and explainability in a single secure workflow.

Comparison with Existing Systems

Feature	GitHub Copilot	JetBrains AI	Docker Tools	Kubernetes Tools	Proposed Solution
Spring Boot Context Awareness	✗	✗	✗	✗	✓
Infrastructure Code Generation	✗	✗	✗	✗	✓
Security Validation	✗	✗	Basic	✗	✓
Multi-layer Explainability (line-by-line)	✗	Partial	✗	✗	✓
IDE Integration	Partial	✓	✗	Partial	Native

Figure 1.1 Comparison with Existing Systems

1. 3 Research Problem

In advancing automation for Spring Boot DevOps workflows, significant challenges persist in generating, securing, and explaining infrastructure artifacts. These challenges highlight the need for context-aware, security-validated, and IDE-native solutions that integrate seamlessly into developer pipelines. The following problems summarize the gaps motivating this research:

Lack of Secure Infrastructure Code Generation

While current approaches such as template-based YAML generators or CI/CD boilerplate repositories accelerate initial setup, they fail to enforce security best practices. Critical issues such as running containers as root, missing resource limits, and unencrypted secrets remain unaddressed [2][4]. This absence of built-in validation results in fragile deployments that expose enterprises to compliance violations and security breaches, demanding a solution that combines automation with rigorous, multi-layer validation.

Lack of Spring Boot–Specific CI/CD Automation

Although Spring Boot has become the most widely adopted backend framework for enterprise Java development [1], most existing automation tools remain framework-agnostic. Jenkins pipelines, GitHub Actions, and scaffolders like JHipster generate general-purpose configurations but neglect Spring Boot–specific conventions such as actuator endpoints, profile

management, and dependency injection patterns. This limits the practical utility of such tools, forcing developers to manually adapt pipelines to Spring Boot's unique runtime requirements.

Limitations of LLM-Only Artifact Generation

Recent advancements in large language models demonstrate their ability to generate YAML manifests, Dockerfiles, and CI/CD scripts. However, studies show that these outputs often suffer from semantic errors, missing configurations, and insecure defaults [3]. LLMs without contextual grounding cannot guarantee alignment with enterprise DevSecOps practices, producing artifacts that compile but fail in production. This highlights the need for hybrid solutions that combine prompt engineering, project context extraction (via MCP), and automated validation rather than relying solely on LLM outputs.

Absence of IDE-Native Integration

Despite the growing popularity of AI coding assistants, there is no IntelliJ-native plugin that integrates Spring Boot project context with secure pipeline generation. Existing assistants like GitHub Copilot focus on source code completion, while Jenkins and ArgoCD orchestrators operate outside the IDE, requiring manual setup. This lack of IDE-native tooling creates workflow fragmentation and prevents secure automation from being seamlessly adopted in real-world Spring Boot projects [12].

Inadequate Support for Project-Specific Context

Current automation frameworks ignore project-specific conventions, such as dependency versions, port configurations, and naming patterns. Without dynamic extraction of project metadata, generated artifacts are often incompatible and require manual patching. The absence of standardized context injection mechanisms, such as the Model Context Protocol (MCP), exacerbates this limitation, reducing trust and slowing developer adoption.

Requirement for Reliability and Explainability

For AI-powered infrastructure assistants to gain enterprise adoption, they must demonstrate both high reliability and transparency. Existing evaluations emphasize productivity metrics but neglect deployment success rates and

explainability quality [7][8]. Without line-by-line justifications for generated code, developers—especially juniors—struggle to understand the rationale behind configurations, undermining trust and learning outcomes. A reliable assistant must not only generate secure, deployable artifacts but also explain its outputs in human-readable terms, fostering confidence and maintainability.

2. OBJECTIVES

2. 1 Main Objectives

The primary objective of this research project is to design and implement an IntelliJ IDEA plugin that leverages Claude 4 Sonnet (via AWS Bedrock), together with live project context extraction through the Model Context Protocol (MCP), to automatically generate secure, optimized, and explainable Dockerfiles, Kubernetes manifests, and CI/CD pipelines for Spring Boot applications.

This objective aims to minimize manual configuration effort, enforce DevSecOps best practices through integrated validation (Hadolint, KubeLinter, and Spring Boot–specific rules), and enhance developer understanding through line-by-line explainability, thereby improving productivity, security, and learning within the Spring Boot ecosystem.

2. 2 Specific Objectives

There are five specific objectives that must be achieved to fulfill the primary objective described above:

1. **Project Context Extraction**

Integrate with the Model Context Protocol (MCP) and abstract syntax tree (AST) analysis to capture live project metadata, including dependencies, configuration files, and runtime profiles of Spring Boot projects.

2. **Artifact Generation via Claude 4 Sonnet**

Utilize Claude 4 Sonnet (via AWS Bedrock) to generate Dockerfiles, Kubernetes manifests, and CI/CD pipeline definitions tailored to Spring Boot applications, ensuring optimized and deployment-ready outputs.

3. **Multi-Layer Security Validation**

Enforce DevSecOps best practices by integrating Hadolint (Docker), KubeLinter (Kubernetes), and custom Spring Boot security checks (non-root execution, resource limits, secret management).

4. **Explainable AI Features**

Develop an explanation engine that provides line-by-line justifications for generated artifacts in clear, beginner-friendly language, supporting developer learning and trust in the system.

5. **Audit and Traceability**

Maintain an audit log of all generated artifacts, validation outcomes, and explanations in PostgreSQL, enabling compliance, reproducibility, and continuous improvement of the assistant.

3. METHODOLOGY

3. 1 System Architecture

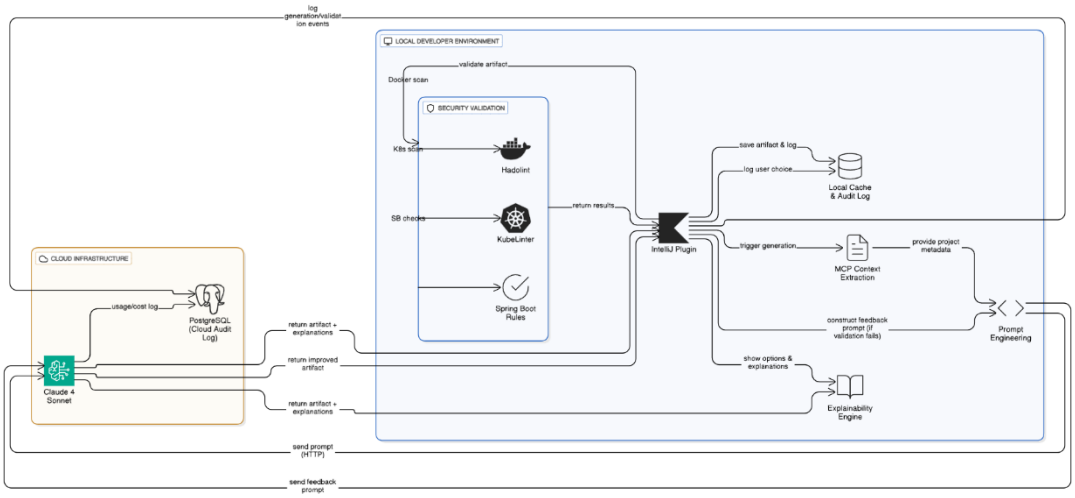

Figure 3.1 Intelligent Spring boot AI Assistant System Architecture

Figure 3.1 illustrates the high-level software architecture of the proposed Intelligent Spring Boot CI/CD Assistant plugin for IntelliJ IDEA. The architecture integrates live project context extraction, Claude 4 Sonnet via AWS Bedrock for artifact generation, multi-layered security validation, explainability features, and enterprise-grade audit logging into a cohesive workflow. The design ensures that Spring Boot developers can generate production-ready DevOps artifacts directly inside their IDE with minimal manual intervention, while maintaining transparency, traceability, and security compliance.

The workflow begins with the developer initiating artifact generation through the IntelliJ plugin interface. This request may target a Dockerfile, a Kubernetes manifest, or a CI/CD pipeline configuration. At this stage, the system automatically analyzes the loaded Spring Boot project using the Model Context Protocol (MCP) and abstract syntax tree (AST) analysis. These components capture rich metadata such as build dependencies (Maven/Gradle), runtime profiles, application properties, port configurations, and any pre-existing containerization descriptors. This metadata ensures that generation is context-aware rather than relying on generic templates.

The extracted project data is then passed into the Prompt Engineering module. This component plays a critical role by packaging the project metadata into structured prompts that specify both functional requirements

(e.g.,"Spring Boot project running on port 8080 with PostgreSQL") and security guidelines (e.g., “must run as non-root, include liveness probes, enforce resource limits”). These structured prompts are transmitted securely to Claude 4 Sonnet via AWS Bedrock, leveraging the model’s long-context reasoning ability and ensuring privacy (as Bedrock guarantees that customer data is not used for training).

Claude 4 Sonnet responds with candidate infrastructure artifacts—such as optimized multi-stage Dockerfiles, Kubernetes manifests with best-practice security contexts, and CI/CD pipeline YAML files tailored to the developer’s environment. These outputs are designed to be syntactically valid, semantically consistent with Spring Boot conventions, and deployment-ready.

Before presenting results to the developer, the system enforces strict checks through the Security Validation Layer. This validation engine integrates multiple tools:

Hadolint performs static analysis of Dockerfiles to detect insecure instructions, missing metadata, or inefficient build practices.

KubeLinter evaluates Kubernetes manifests for compliance with production standards, ensuring safe defaults for probes, secrets, and RBAC.

Spring Boot Rules apply framework-specific checks, such as enforcing externalized configuration, preventing hardcoded secrets, and verifying dependency management.

If validation reveals issues, the feedback loop triggers prompt refinement—automatically adjusting the input to Claude and re-generating corrected artifacts. This ensures that outputs provided to developers already conform to DevSecOps best practices.

Once validated, the artifacts are enriched through the Explainability Engine. This module generates line-by-line explanations of the produced files in natural language, clarifying the intent, best-practice rationale, and security implications of each configuration directive. This not only improves trust in the system but also transforms it into an educational assistant, helping junior developers understand infrastructure code while using it.

The final outputs, along with validation results and explanations, are presented back to the developer within IntelliJ IDEA for review and adoption. All interactions are also captured in the Audit & Telemetry subsystem, which records generation events, validation outcomes, developer selections, and usage statistics into a PostgreSQL database. These audit logs support traceability, compliance, and research evaluation while also enabling long-term analytics such as average setup time saved or most common validation failures.

This architectural design emphasizes a balance between automation and transparency. Developers can focus on application logic while the system automates complex DevOps configurations, but without turning into a “black box.” The use of MCP-driven context extraction ensures accuracy, Claude 4 Sonnet provides intelligence and adaptability, and validation plus

explainability modules guarantee trustworthiness. Together, these layers deliver artifacts that are not only functional and secure but also educationally valuable, positioning the plugin as both a productivity booster and a knowledge-transfer tool.

3. 2 Software Solution

The Intelligent Spring Boot CI/CD Assistant is implemented as an IntelliJ IDEA plugin that integrates Claude 4 Sonnet (via AWS Bedrock) with project context extraction, security validation, explainability, and audit logging. The solution ensures that developers can generate Dockerfiles, Kubernetes manifests, and CI/CD pipelines that are production-ready, secure, and easy to understand.

1. System Architecture and Workflow

The architecture is modular to support scalability and maintainability. The main components are:

- Project Context Extraction Module
- Prompt Engineering & AI Artifact Generation Engine
- Multi-Layer Security Validation Framework
- Explainability and Developer Guidance Module
- IDE Native Integration Layer
- Audit and Telemetry System

These components interact in a continuous workflow, triggered within IntelliJ IDEA whenever the developer requests infrastructure artifact generation.

2. Project Context Extraction

The first step is to accurately extract project context.

- Model Context Protocol (MCP) ensures real-time extraction of build dependencies, application profiles, and configurations.
- AST (Abstract Syntax Tree) Analysis scans Spring Boot code and YAML files to capture annotations, beans, and environment-specific parameters.
- Metadata Aggregation compiles this information into structured project metadata, forming the basis for artifact generation.

3. AI-Based Artifact Generation Engine

At the core lies Claude 4 Sonnet, which generates infrastructure code tailored to the extracted metadata.

- Prompt Engineering packages project metadata and security constraints into structured prompts.

- Artifacts Generated include:

Dockerfiles → multi-stage builds, non-root execution, health checks.
Kubernetes manifests → deployments, services, secrets, resource limits.
CI/CD pipelines → YAML workflows optimized for Maven/Gradle builds.

- Feedback Loop: If validation fails, prompts are refined and re-sent until compliance is achieved.

4. Multi-Layer Security Validation Framework

- Security validation ensures that all generated artifacts comply with DevSecOps standards.
- Hadolint checks Dockerfiles for insecure instructions and poor practices.
- KubeLinter validates Kubernetes manifests against CNCF security guidelines.

- Spring Boot Rules enforce secure defaults such as externalized configuration, resource limits, and safe profile handling.
- Continuous Validation ensures real-time feedback to developers during artifact generation.

5. Explainability and Developer Guidance

The assistant is designed to be transparent and educational.

- Line-by-Line Explanations describe the purpose and security implications of each configuration.
- Interactive Queries allow developers to ask “why” a certain configuration was generated.
- Learning Resources link to Spring Boot documentation and DevOps guides for further study.

6. IDE Native Integration Layer

Seamless integration within IntelliJ IDEA ensures minimal disruption.

- Plugin Development with Kotlin provides UI elements like side panels and pop-ups.
- Project-Aware Invocation keeps artifact suggestions updated with project changes.
- Real-Time Feedback shows validation errors inline.
- Git Integration tracks generated artifacts for rollback and version control.

7. Audit and Telemetry System

PostgreSQL is used for secure logging and analytics.

- Audit Logs capture generated artifacts, validation results, and developer actions for compliance.

- Analytics Dashboard measures time saved, validation success rates, and developer adoption.
- Feedback Loop incorporates developer evaluations to continuously improve performance.

8. Development Methodology

The plugin development follows Agile SDLC principles.

- Iterative Sprints deliver incremental features with developer feedback.
- Automated Testing covers unit, integration, and validation checks.
- UX-Driven Design ensures usability for both juniors and experienced developers.
- Security by Design applies dependency scans and penetration testing throughout the lifecycle.

This detailed methodology ensures the *Intelligent Spring Boot CI/CD Assistant* is a state-of-the-art, context-aware, and secure automation tool that seamlessly enhances the developer experience while addressing critical needs in Spring Boot infrastructure generation and security compliance.

4. PROJECT REQUIREMENTS

4. 1 Functional Requirements

1. Context Extraction

The system must analyze the structure of a Spring Boot project, including build tools (Maven/Gradle), configuration files (e.g., application.yml), and declared dependencies, to gather the metadata required for artifact generation.

2. Dockerfile Generation

The assistant must generate optimized Dockerfiles that follow best practices such as multi-stage builds, non-root user execution, and inclusion of application health checks for production readiness.

3. Kubernetes Manifests

The system must produce deployment, service, and configuration YAMLs with built-in security hardening, resource constraints, and compliance with container orchestration standards.

4. CI/CD Pipelines

The assistant must generate continuous integration and deployment pipeline configurations for popular tools such as GitHub Actions, GitLab CI, and Jenkins, enabling automated build, test, and deployment workflows.

5. Security Validation

All generated artifacts must be validated through Hadolint (for Dockerfiles), KubeLinter (for Kubernetes manifests), and custom Spring Boot–specific security rules to ensure compliance with DevSecOps standards.

6. Explanations

The system must provide detailed, line-by-line natural language explanations of generated files, helping developers, especially junior team members, understand the rationale behind best practices applied.

7. Audit Logging

The assistant must log all generated outputs, validation results, and explanations into a secure PostgreSQL database to maintain audit trails, facilitate analytics, and support compliance monitoring.

8. IDE Integration

The solution must be seamlessly integrated into IntelliJ IDEA, providing a native interface with chat-style interaction, generation buttons, and explanation panels, ensuring a smooth developer workflow without external tools.

4. 2 Non-functional requirements

1. **Performance -** Response time for artifact generation: ≤ 2 seconds , Support 100+ concurrent requests.
2. **Availability –** System uptime must be 99% and cloud deployment with horizontal scaling.
3. **Security -** Encryption: TLS for data in transit, AES for audit logs at rest, Role-based access for enterprise deployments**.**
4. **Usability-** Learning curve: ≤30 minutes for new users, WCAG 2.1 AA accessibility compliance, Multilingual-ready (starting with English).
5. **Maintainability-**Modular design for adding new validators (e.g., Terraform, Helm charts), Automated testing (unit + integration).

4. 3 System requirements

The purpose of software requirements is to specify the necessary resources that a system must implement to ensure the proper functioning of the proposed system. The software specification requirements for this component are outlined as follows.

- **IDE Plugin:** Java, IntelliJ SDK 2024.3+
- **LLM Service:** Claude 4 Sonnet (via AWS Bedrock)
- **Validation Tools:** Hadolint v2.12+, KubeLinter v0.6+
- **Database:** PostgreSQL 15
- **Context Extraction**: Model Context Protocol (MCP)
- **Deployment:** Docker + Kubernetes

4. 4 User requirements

This plugin is mainly developed for two types of users.

1. Enterprise Spring Boot Development Teams

Professional development teams working on Spring Boot applications who need to streamline their DevOps workflows through automated infrastructure generation. These users require Automated creation of production-ready Dockerfiles, Kubernetes manifests, and CI/CD pipelines. Security-compliant infrastructure artifacts that meet enterprise standards. Reduced time spent on manual DevOps configuration and deployment setup
Integration with existing development workflows within IntelliJ IDEA.

2. DevOps Engineers and Platform Teams

DevOps professionals responsible for establishing and maintaining deployment pipelines for Spring Boot applications. They benefit from Standardized, secure infrastructure templates across multiple projects, Automated security validation and compliance checking, Consistent deployment patterns and best practices enforcement, Audit trails and analytics for infrastructure governance

3. Junior Developers and DevOps Newcomers

Developers who are proficient in Spring Boot application development but lack extensive experience with containerization, Kubernetes, and CI/CD pipeline configuration. The plugin serves as an educational tool by providing explainable infrastructure code with line-by-line justifications, teaching DevSecOps best practices through generated examples, Reducing the learning curve for complex deployment technologies, Offering contextual guidance and documentation links

5. Gantt Chart

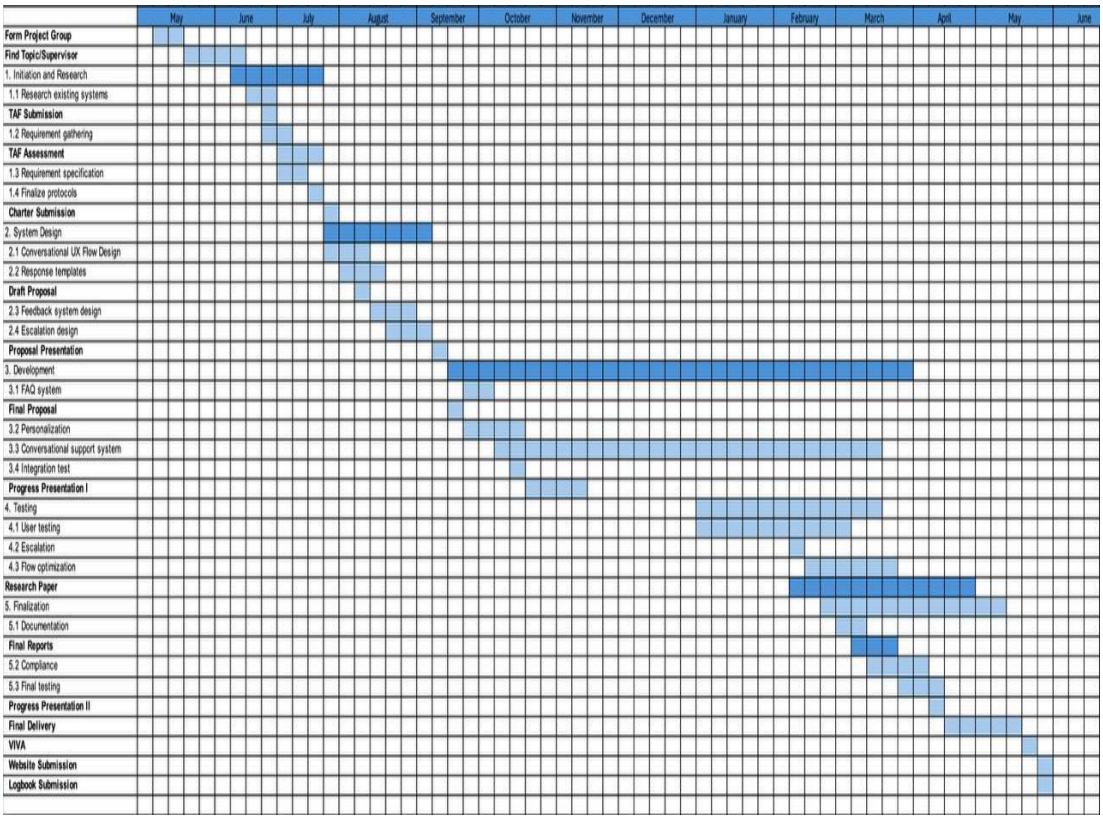

Figure 5.1:Gantt Chart

5. 1 Work Breakdown Structure (WBS)

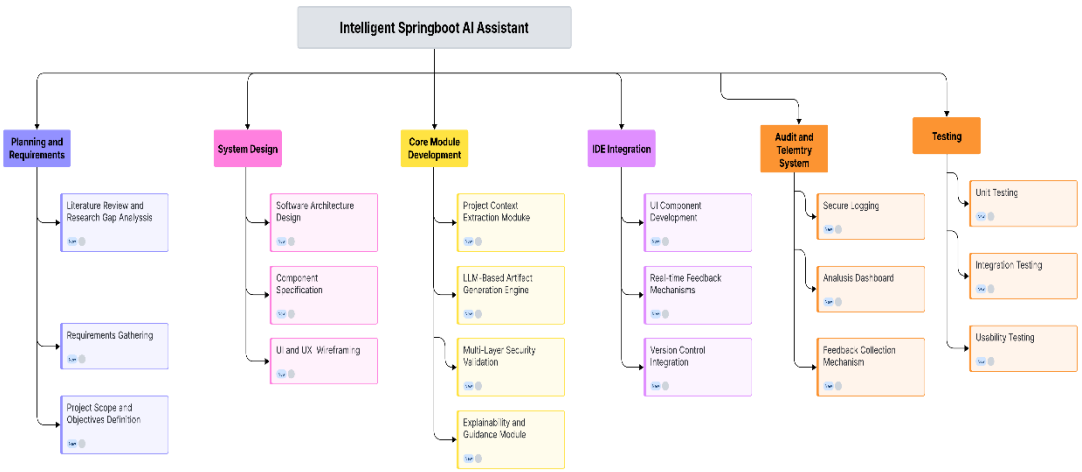

Figure 5.1 Work Breakdown Structure

6. BUDGET AND BUDGET JUSTIFICATION

The Table 6.1 below depicts the overall budget estimation for the proposed IntelliJ IDEA plugin system, including costs for dataset preparation, model training, deployment, and operational expenses.

Requirement	**Cost (LKR)**
Cost for data collection and preprocessing (GitHub projects, curation, annotation)	0.00 (Using publicly available spring boot repositories to curate the dataset manually)
Internet Charges	Rs. 5000
LLM Service (Claude 4 Sonnet via AWS Bedrock)	0.00 (within AWS Free Tier – $200 credits)
Total Cost	Rs. 5000

Table 6.1 Budget Justification table

REFERENCES

[1] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, et al., ‘Evaluating Large Language Models Trained on Code,” arXiv preprint arXiv:2107.03374, 2021.

[2] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, “CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation,” in Proceedings of EMNLP 2021, pp. 8690–8705, 2021.

[3] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, “Do Users Write More Insecure Code with AI Assistants?” in Proceedings of the 2023 ACM Conference on Computer and Communications Security (CCS), pp. 1–18, 2022.

[4] S. Kumar, R. Raj, and A. Agrawal, “Infrastructure as Code Security: A Systematic Literature Review,” IEEE Transactions on Software Engineering, vol. 49, no. 2, pp. 560–578, 2022.

[5] H. Zhang, L. Yu, and M. Chen, “Deployability-Centric IaC Generation,” in Proceedings of the International Conference on Software Engineering (ICSE), pp. 115–128, 2025.

[6] Meta AI, “Code LLaMA: Open Foundation Models for Code,” Meta Research Blog, Aug. 2023. [Online]. Available: https://ai.meta.com/research/publications/code-llama

[7] Meta AI,“Introducing LLaMA 3 Models,” Meta Research Blog, Apr. 2025. [Online]. Available: https://ai.meta.com/research/publications/llama-3

[8] A. Touvron, H. Martin, K. Stone, et al., “LLaMA 2: Open Foundation and Fine-Tuned Chat Models,” arXiv preprint arXiv:2307.09288, 2023.

[9] H. Waheed, A. Iqbal, and M. R. Gunawardena, “Error Metrics and Performance Fitness Indicators for AI/ML in Software Engineering,” Architecture, Structures and Construction, vol. 5, pp. 112–126, 2023.

[10] R. Coppola, “AI-based Code Generation Framework,” Journal of Software Engineering Research and Development, vol. 12, no. 3, pp. 65–78, 2024.

[11] OWASP Foundation, “LLM Prompt Injection Prevention Cheat Sheet,” OWASP Documentation, 2024. [Online]. Available: https://owasp.org

[12] Docker Inc., “Docker Security Best Practices,” Docker Documentation, 2024. [Online]. Available: https://docs.docker.com/develop/security/

[13] Cloud Native Computing Foundation (CNCF), “Kubernetes Security Best Practices,” CNCF Whitepaper, 2024. [Online]. Available: https://kubernetes.io/docs/concepts/security/

[14] Hadolint Contributors, “Hadolint – Dockerfile Linter for Security Best Practices,” GitHub Repository, 2024. [Online]. Available: https://github.com/hadolint/hadolint

[15] StackRox (Red Hat),“KubeLinter: Kubernetes YAML Validation Tool,” Documentation, 2024. [Online]. Available: https://docs.kubelinter.io

[16] Spring Boot Team,“Spring Boot Security Configuration Guide,” Spring Documentation, 2024. [Online]. Available: https://docs.spring.io/spring-boot/docs/current/reference/html/security.html

[17] JetBrains,“AI Assistant in IntelliJ IDEA,” JetBrains Blog, Feb. 2025. [Online]. Available: https://blog.jetbrains.com

[18] Gartner,“DevOps Market Report 2023–2028,” Gartner Research, 2024.

[19] DZone,“Best Practices for Building Secure CI/CD Pipelines,” DZone DevOps Guide, 2024.

[20] Trivy Project,“Trivy: A Comprehensive Vulnerability Scanner for Containers,” GitHub, 2024.